

ระบบฐานข้อมูลขั้นสูง | สัปดาห์ที่ 1

Assignment 1

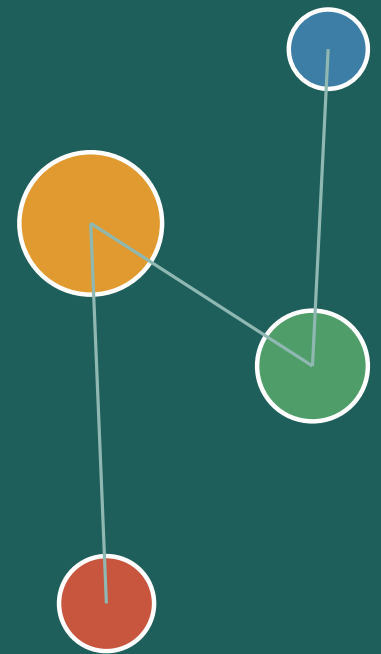
กิจกรรมในชั้นเรียน และการวิเคราะห์ Workload เพื่อ
เลือกเทคโนโลยีฐานข้อมูล

จัดทำโดย

นายจิรภัทร จันทรัมย์

รหัสนักศึกษา 664245026 หมู่เรียน 66/43

มหาวิทยาลัยราชภัฏนครปฐม



หัวข้อในงานนี้

ทำตามกิจกรรมในหน้า "กิจกรรมในชั้นเรียนและงานมอบหมาย" ของสัปดาห์ที่ 1 ครบทั้ง 4 ข้อ

1

กิจกรรม 1

เลือก DB ให้เหมาะกับ Workload

Banking, Social Network, Movie Recommendation, Product Catalog, AI Chatbot

2

กิจกรรม 2

Table หรือ Graph?

ออกแบบ Somchai → Somsri → Mana → Nida ทั้งแบบตารางและแบบกราฟ

3

กิจกรรม 3

ออกแบบ Social Graph

สำหรับ Mini Project: User, Post, News, Source, Hashtag

4

Assignment 1

วิเคราะห์ Application 3 ระบบ

ลักษณะข้อมูล, Workload / Access Pattern, เทคโนโลยีที่เหมาะสม และ Polyglot

กิจกรรม 1: เลือก Database ให้เหมาะกับ Workload

ระบบ	Workload / Access Pattern ที่เด่น	ฐานข้อมูลที่เลือก	เหตุผล
Banking	เขียนบ่อยและต้องถูกต้อง 100% โอนเงินต้องสำเร็จทั้งชุดหรือยกเลิกทั้งชุด	Relational (PostgreSQL, Oracle)	มี ACID transaction, constraint และ foreign key ช่วยกัน ยอดเงินผิด
Social Network	ใครติดตามใคร, หาเพื่อนของเพื่อน (เดินตามความสัมพันธ์หลายชั้น)	Graph (Neo4j)	relationship เป็นข้อมูลหลัก traversal หลาย hop ได้โดยไม่ต้อง JOIN ซ้ำ ๆ
Movie Recommendation	user ดู/ให้คะแนนหนัง → หาคนที่ชอบคล้ายกัน → แนะนำเรื่องที่เขาชอบ	Graph (Neo4j) + Vector Search	path User→Movie→User→Movie คือการแนะนำโดยตรง ส่วน vector ใช้หาหนังที่เนื้อหาคล้ายกัน
Product Catalog	อ่านมากกว่าเขียน สินค้าแต่ละประเภทมี field ไม่เหมือนกัน	Document (MongoDB)	schema ยืดหยุ่น อ่านข้อมูลสินค้าทั้งก้อนได้ในครั้งเดียว
AI Chatbot	ค้นเอกสารตามความหมาย (RAG) และเก็บ session/ประวัติแชท	Vector Search + Key-Value (Redis)	similarity search หาเนื้อหาที่ความหมายใกล้เคียงกัน, Redis อ่าน/เขียน session เร็ว

ข้อสังเกต: Movie Recommendation กับ AI Chatbot ใช้ฐานข้อมูลมากกว่า 1 แบบ ซึ่งก็คือแนวคิด Polyglot Persistence

กิจกรรม 2: Table หรือ Graph?

โจทย์: Somchai follows Somsri → Mana → Nida ลองออกแบบข้อมูลชุดเดียวกัน 2 แบบ

แบบตาราง (Relational)

users

user_id (PK)	name
U01	Somchai
U02	Somsri
U03	Mana
U04	Nida

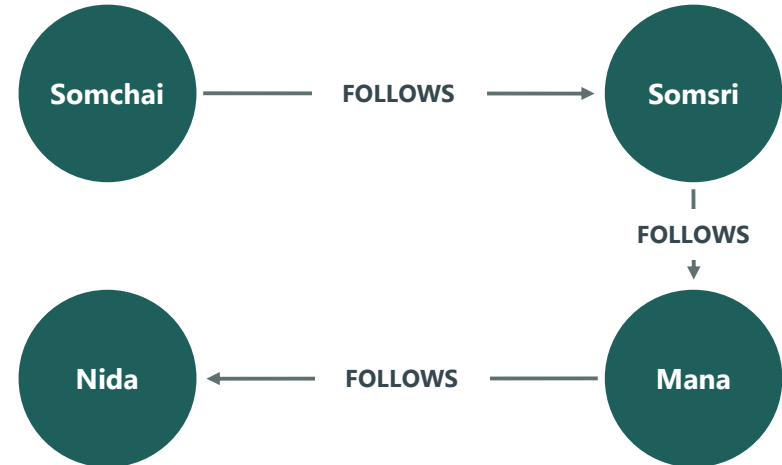
follows

follower_id	followee_id
U01	U02
U02	U03
U03	U04

follower_id และ followee_id เป็น FK อ้างอิงไปที่ users.user_id

- 1 แถวใน follows = ความสัมพันธ์ 1 เส้น
- ต้องใช้ 2 ตาราง ความสัมพันธ์ถูกเก็บผ่านคีย์
- อยากรู้ว่าใครตามใครต่อกันกี่ชั้น ต้อง JOIN เอง

แบบกราฟ (Property Graph)



(:User {user_id:'U01', name:'Somchai'})

- ข้อมูลชุดเดิม แต่ความสัมพันธ์กลายเป็นเส้นเชื่อมโดยตรง
- Node = User, Relationship = FOLLOWS (มีทิศทาง)

กิจกรรม 2: ลอง Query ว่า Somchai เชื่อมไปถึง Nida ยังไง

SQL (ต้อง JOIN follows 1 ครั้งต่อ 1 hop)

```
SELECT a.name, b.name, c.name, d.name
FROM follows f1
JOIN follows f2 ON f1.followee_id = f2.follower_id
JOIN follows f3 ON f2.followee_id = f3.follower_id
JOIN users a ON a.user_id = f1.follower_id
JOIN users b ON b.user_id = f1.followee_id
JOIN users c ON c.user_id = f2.followee_id
JOIN users d ON d.user_id = f3.followee_id
WHERE a.name = 'Somchai'
      AND d.name = 'Nida';
```

Cypher (Neo4j)

```
MATCH p = (a:User {name:'Somchai'})
          -[:FOLLOWS*1..3]->
          (b:User {name:'Nida'})
RETURN [n IN nodes(p) | n.name]
       AS path;
```

ผลลัพธ์

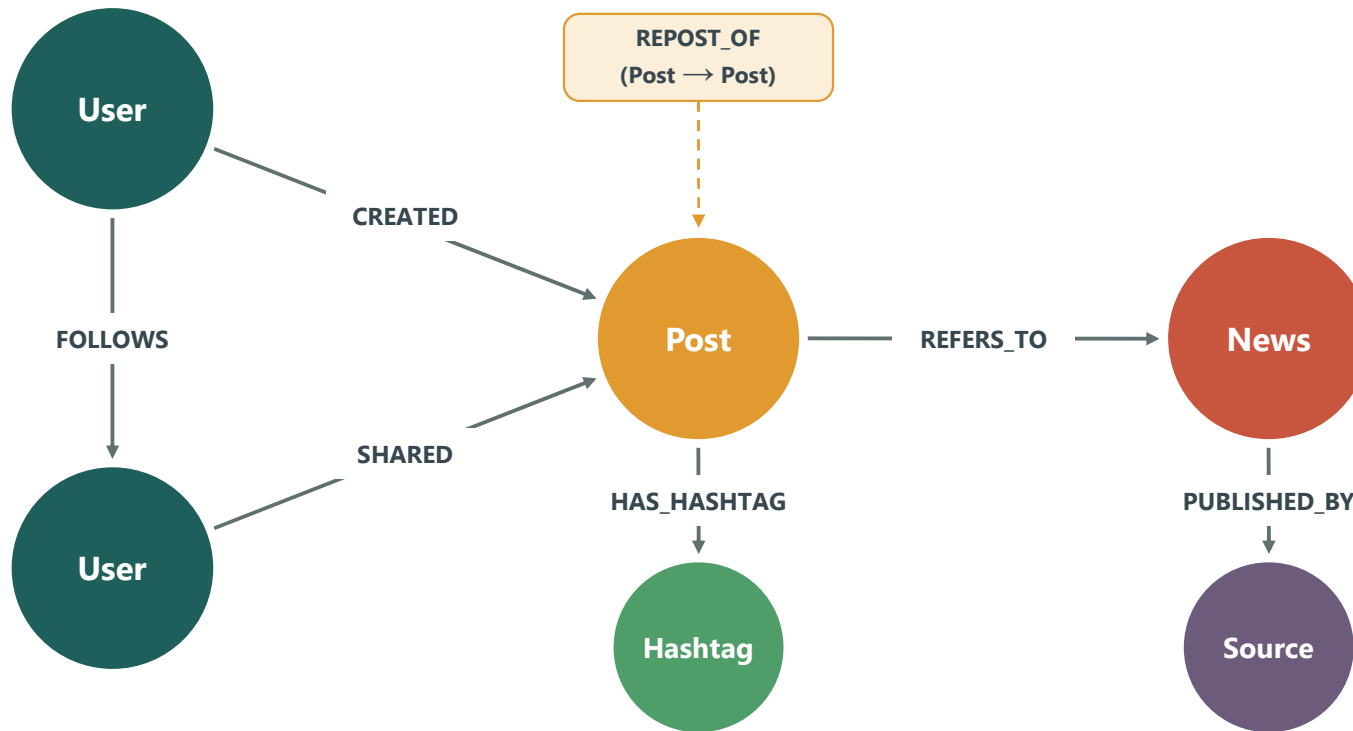
```
[Somchai, Somsri, Mana, Nida]
```

สรุปที่ได้จากกิจกรรมนี้

- SQL ทำได้ แต่ถ้าไม่รู้ว่าจะหากล่อง hop ต้องเขียน recursive CTE และยิงหลายชั้นยิ่ง JOIN เยอะ
- Cypher เขียนเป็น pattern ตรง ๆ อยากค้นลึกขึ้นแค่เปลี่ยน *1..3 เป็น *1..6
- ข้อมูล 4 คนแบบนี้ใช้ตารางก็พอ แต่ถ้าคำถามหลักของระบบคือ "เส้นทาง" กราฟจะเหมาะกว่า

กิจกรรม 3: ออกแบบ Social Graph สำหรับ Mini Project

คำถามตั้งต้น: ข่าวหนึ่งเรื่องถูกส่งต่อจากผู้ใช้กลุ่มใดไปกลุ่มใด และใครเป็นตัวกลางสำคัญ



สรุปโมเดล

5

Node label

7

Relationship type

เก็บเวลาไว้ที่ Post และ SHARED เพื่อดูลำดับการแพร่ข่าว

กิจกรรม 3: Node, Property และ Relationship

Node และ Property

Label	Property หลัก
User	user_id, name, community
Post	post_id, text, created_at
News	news_id, title, label (จริง/ปลอม/ยังไม่ยืนยัน)
Source	source_id, domain, type
Hashtag	name

Relationship

```
(User)-[:FOLLOWS {since}]->(User)
(User)-[:CREATED]->(Post)
(User)-[:SHARED {shared_at}]->(Post)
(Post)-[:REPOST_OF]->(Post)
(Post)-[:REFERS_TO]->(News)
(Post)-[:HAS_HASHTAG]->(Hashtag)
(News)-[:PUBLISHED_BY]->(Source)
```

เหตุผลที่ออกแบบแบบนี้

- แยก Post ออกจาก News เพราะข่าวเดียวกันถูกหลายโพสต์อ้างถึงได้ จะได้นับว่าข่าวนี้แพร่ไปกี่โพสต์
- ใส่ created_at / shared_at ไว้ด้วย เพราะดูแค่ใครเชื่อมกับใครไม่พอ ต้องรู้ลำดับเวลาถึงจะดูเส้นทางแพร่ข่าวได้

กิจกรรม 3: คำถามที่ Social Graph นี้ตอบได้

1 ข่าวไหนถูกแชร์โดยคนมากที่สุด

```
MATCH (u:User)-[:SHARED]->(:Post)
      -[:REFERS_TO]->(n:News)
RETURN n.title,
       count(DISTINCT u) AS sharers
ORDER BY sharers DESC LIMIT 5;
```

2 ข่าวไหนแพร่ข้ามหลาย community

```
MATCH (u:User)-[:SHARED]->(:Post)
      -[:REFERS_TO]->(n:News)
RETURN n.title,
       count(DISTINCT u.community)
       AS groups
ORDER BY groups DESC;
```

3 ใครเป็นตัวกลางของเครือข่าย

```
CALL gds.betweenness.stream(
  'socialGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId)
       .name AS user, score
ORDER BY score DESC LIMIT 5;
```

ข้อควรระวังตอนสรุปผล

- ข้อ 3 ใช้ Graph Data Science (betweenness centrality) ต้อง project กราฟก่อนถึงจะ CALL ได้
- ผู้ใช้ที่โพสต์เร็วสุดใน dataset ไม่จำเป็นต้องเป็น "ต้นกำเนิดจริง" ของข่าว ต้องดู timestamp และแหล่งที่มา (provenance) ประกอบ

Assignment 1: ระบบที่เลือกมาวิเคราะห์

เลือก Application 3 ระบบ แล้ววิเคราะห์ตามหัวข้อ: ลักษณะข้อมูล, Workload / Access Pattern, เทคโนโลยีที่เหมาะสม และ Polyglot

1

ระบบลงทะเบียนเรียน

ของมหาวิทยาลัย

ทำไมถึงเลือก: ใช้อยู่ทุกเทอม และเห็นปัญหาตอนเปิดลงทะเบียนที่คนเข้าพร้อมกันเยอะ ๆ

2

แอปสั่งอาหาร

Food Delivery

ทำไมถึงเลือก: มีข้อมูลหลายแบบในระบบเดียว ทั้งออเดอร์ เมนู และตำแหน่งไรเดอร์แบบ real-time

3

Social Classroom

โปรเจกต์จบของผม

ทำไมถึงเลือก: เป็นระบบที่กำลังพัฒนาอยู่ เลยอยากลองวิเคราะห์ว่าถ้าขยายระบบควรใช้ฐานข้อมูลแบบไหน

เกณฑ์ที่ใช้เป็นแนวในการวิเคราะห์: วิเคราะห์ข้อมูล 3 | Workload 3 | เลือกเทคโนโลยี 2 | Polyglot 1 | ความชัดเจน 1

ระบบที่ 1: ระบบลงทะเบียนเรียนของมหาวิทยาลัย

ระบบที่นักศึกษาใช้ลงทะเบียนรายวิชา ดูตารางเรียน ผลการเรียนรู้ และชำระค่าเทอม

1 ลักษณะข้อมูล

- **Structured เป็นหลัก:** นักศึกษา, รายวิชา, กลุ่มเรียน (section), อาจารย์, การลงทะเบียน, เกรต
- Entity สัมพันธ์กันชัดเจนผ่านคีย์ เช่น student_id, section_id
- **Semi-structured:** log การเข้าใช้ระบบ
- **Unstructured:** ไฟล์ใบเสร็จ, เอกสาร PDF

2 Workload / Access Pattern

- ช่วงเปิดลงทะเบียน write พุ่งสูงมาก นักศึกษาหลายคนแย่งที่นั่งวิชาเดียวกันพร้อมกัน
- ที่นั่งห้ามเกินจำนวนที่เปิด (ต้อง consistent)
- นอกช่วงลงทะเบียนเป็น read เกือบทั้งหมด เช่น ดูตารางเรียน ดูเกรด
- Access pattern: lookup ด้วย student_id, เช็คนั่งตาม section_id, JOIN หลายตารางเพื่อออกรายงาน

3 เทคโนโลยีที่เลือก + เหตุผล

- **Relational (MySQL InnoDB / PostgreSQL)**
- ACID transaction ลงทะเบียน + ตัดที่นั่งต้องสำเร็จพร้อมกัน
- ใช้ row lock (SELECT ... FOR UPDATE) และ CHECK constraint กันที่นั่งเกิน
- ข้อมูลเป็นตาราง JOIN ออกรายงานได้ดี

4 Polyglot: Relational เป็นฐานข้อมูลหลัก + Redis cache จำนวนที่นั่งคงเหลือให้หน้าเว็บอ่านเร็วช่วง peak + Object Storage เก็บไฟล์เอกสาร

ระบบที่ 2: แอปส่งอาหาร (Food Delivery)

ลูกค้าสั่งอาหารจากร้าน ระบบหาไรเดอร์ใกล้เคียง ๆ และติดตามตำแหน่งแบบ real-time จนส่งถึง

1 ลักษณะข้อมูล

- **Structured:** ออเดอร์, การชำระเงิน, ลูกค้า, ร้าน, ไรเดอร์
- **Semi-structured:** เมนูแต่ละร้านมี option ไม่เหมือนกัน (ที่อปปิ้ง, ระดับความหวาน/เผ็ด) เหมาะกับ JSON
- **Geo / real-time:** พิกัดไรเดอร์ อัปเดตทุกไม่กี่วินาที
- **Unstructured:** รูปภาพอาหาร, รีวิว

2 Workload / Access Pattern

- เขียนตำแหน่งไรเดอร์ถี่มาก (write สูง) แต่ใช้แค่ค่าล่าสุด
- Peak ช่วงช่วงมือเที่ยงและเย็น
- อ่านเมนู/ร้านเยอะกว่าเขียนหลายเท่า
- ค้นหา "ร้านใกล้ฉัน" และ "ไรเดอร์ใกล้ร้าน" (geo query)
- ออเดอร์ + การชำระเงินต้องถูกต้อง ห้ามตัดเงินซ้ำ

3 เทคโนโลยีที่เลือก + เหตุผล

- **Relational:** ออเดอร์, payment (ต้องการ ACID)
- **Document (MongoDB):** ร้านและเมนูที่ field ยืดหยุ่น
- **Key-Value (Redis GEO):** ตำแหน่งไรเดอร์ล่าสุด อ่าน/เขียนเร็วใน memory และหาระยะใกล้ได้
- **Object Storage:** รูปภาพอาหาร

4 Polyglot: ระบบนี้เป็น Polyglot ชัดที่สุด ใช้ 4 แบบตามส่วนของระบบ (ดูแผนภาพในหน้าถัดไป)

ระบบที่ 3: Social Classroom (โปรเจกต์จบ)

เว็บแอปห้องเรียนแบบโซเชียล อาจารย์โพสต์ประกาศ/งาน นักศึกษาคอมเมนต์ ส่งงาน และดูคะแนน

1 ลักษณะข้อมูล

- **Structured:** ผู้ใช้ (อาจารย์/นักศึกษา), ห้องเรียน, งานที่มอบหมาย, การส่งงาน, คะแนน
- **Semi-structured:** โพสต์และคอมเมนต์ (มีไฟล์แนบ, reaction, mention)
- **Unstructured:** ไฟล์งาน, รูป, วิดีโอ
- มีความสัมพันธ์แบบโซเชียล: สมาชิกห้อง, กลุ่มงาน, เรียนวิชาเดียวกัน

2 Workload / Access Pattern

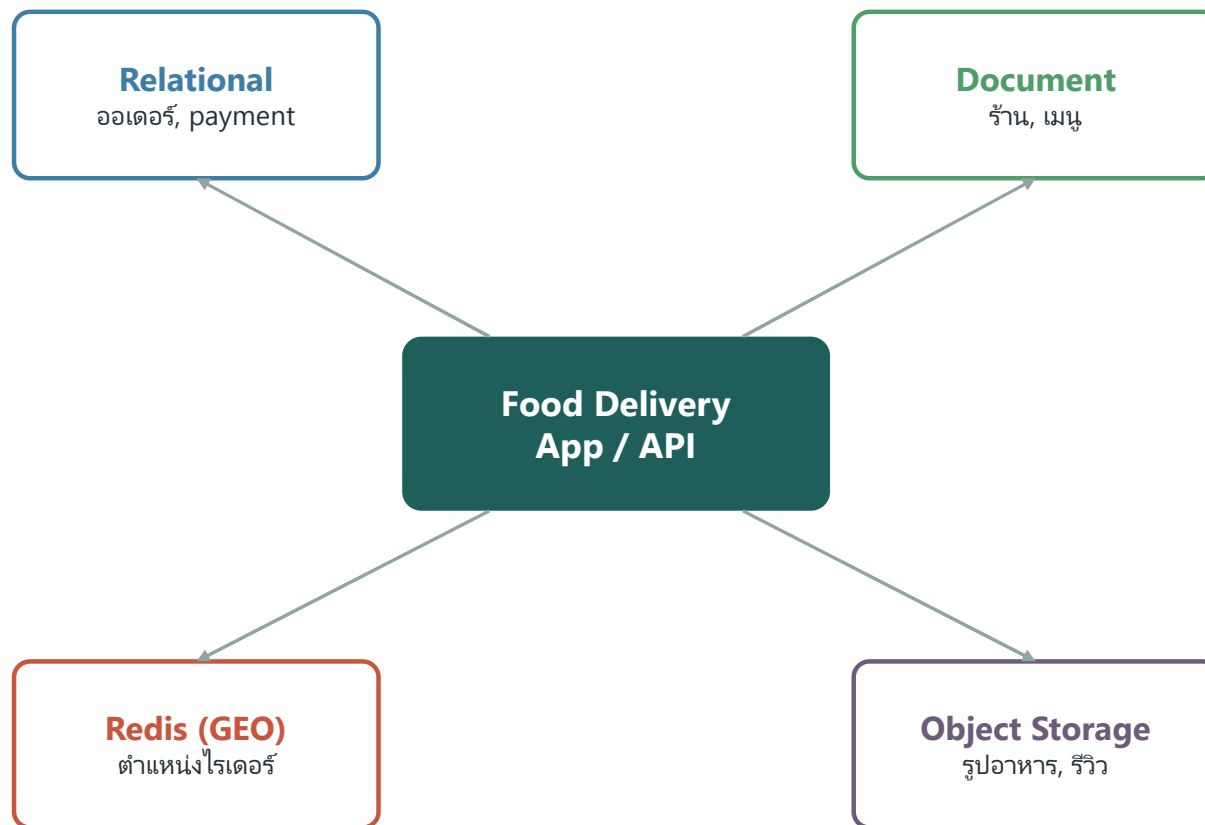
- Read-heavy: เปิดดูฟีดของห้องเรียนบ่อยที่สุด
- ใกล้ deadline มีการส่งงานพร้อมกันจำนวนมาก
- คะแนนต้องถูกต้อง แก้แล้วต้องเห็นตรงกัน
- Access pattern: ดึงฟีดตาม class_id เรียงตามเวลา, หางานที่ยังไม่ส่งของ student_id, แนะนำเพื่อน/กลุ่มที่เรียนวิชาเดียวกัน

3 เทคโนโลยีที่เลือก + เหตุผล

- **Relational (หลัก):** users, classes, assignments, grades ต้องการ ACID
- **Document / JSONB:** โพสต์และคอมเมนต์ที่โครงสร้างไม่ตายตัว
- **Graph (ต่อยอด):** แนะนำเพื่อนร่วมกลุ่มจากความสัมพันธ์หลายชั้น
- **Object Storage + Redis:** ไฟล์งาน และ notification / session

4 **Polyglot:** ช่วงแรกใช้ Relational ตัวเดียว (เก็บโพสต์เป็น JSONB) ก็พอ ถ้าผู้ใช้เยอะขึ้นค่อยแยก Graph และ Redis ออกมา ไม่ต้องใช้หลาย DB ตั้งแต่แรก

ตัวอย่าง Polyglot Persistence: แอปสั่งอาหาร



ข้อดี

- แต่ละส่วนได้ DB ที่ตรงกับ workload
- scale แยกส่วนได้ เช่น เพิ่ม Redis ช่วง peak

ข้อแลกเปลี่ยน

- ต้อง integrate และ monitor หลายระบบ
- ข้อมูลข้าม DB อาจไม่ consistent ทันที
- ทีมต้องรู้หลายเทคโนโลยี และต้องดูแลเรื่อง security ทุกตัว

สรุปเปรียบเทียบ 3 ระบบ

ระบบ	ข้อมูลเด่น	Workload เด่น	DB หลัก	DB เสริม (Polyglot)
ลงทะเบียนเรียน	Structured, คีย์สัมพันธ์ชัด	write พร้อมกันสูงช่วงเปิดลงทะเบียน, ต้อง consistent	Relational	Redis (cache), Object Storage
แอปสั่งอาหาร	หลากหลาย: ตาราง, JSON, พิกัด, รูป	write ตำแหน่งถี่, geo query, peak ตามมืออาหาร	Relational + Document	Redis GEO, Object Storage
Social Classroom	Structured + โพสต์/คอมเมนต์ + ไฟล์	read-heavy (ฟีด), ส่งงานพร้อมกันใกล้ deadline	Relational (JSONB)	Graph, Redis, Object Storage

สิ่งที่สังเกตได้

- ทั้ง 3 ระบบยังต้องมี Relational เป็นแกนกลาง เพราะมีส่วนที่ต้องการ transaction (เกรด, ที่นั่ง, การจ่ายเงิน)
- จำนวน DB ที่ใช้ขึ้นกับความหลากหลายของข้อมูลและ workload ไม่ใช่वाईงใช้หลายตัวยิ่งดี

สรุปสิ่งที่ได้เรียนรู้

1

เริ่มจาก Access Pattern

ก่อนเลือก DB ต้องถามก่อนว่าระบบอ่าน/เขียนข้อมูลแบบไหน
ไม่ใช่เลือกจากความคุ้นเคย

2

SQL ทำได้ แต่ Graph อาจเหมาะกว่า

งานที่ต้องเดินตามความสัมพันธ์หลายชั้น เช่น follow หรือการ
แพร่ข่าว กราฟเขียนและอ่านง่ายกว่า

3

Polyglot ต้องคุ้มค่า

ใช้หลายฐานข้อมูลได้ แต่ต้องแลกกับความซับซ้อนในการดูแล
ระบบ

4

เวลาสำคัญกับการวิเคราะห์ข่าว

การหาเส้นทางแพร่ข่าวต้องมี timestamp และ provenance
ไม่ใช่แค่การเชื่อมต่อ

เอกสารอ้างอิง

- เอกสารประกอบการสอน รายวิชาการระบบฐานข้อมูลขั้นสูง สัปดาห์ที่ 1: แนวคิดและสถาปัตยกรรมของระบบฐานข้อมูลขั้นสูง
- Elmasri, R. & Navathe, S. B. Fundamentals of Database Systems. Pearson.
- Silberschatz, A., Korth, H. F. & Sudarshan, S. Database System Concepts. McGraw-Hill.
- Neo4j Documentation: Graph database concepts, Cypher Manual, Graph Data Science Library
- MongoDB Documentation: Data Modeling
- Redis Documentation: Data types (Strings, Hashes, Geospatial)
- PostgreSQL Documentation: Transactions, Explicit Locking, JSON Types